Recommender System

Dobrosława Hetmańczyk

May 13, 2026

Contents

1 Introduction

The key objective of this project is to create a recomendation system for movie ratings using four different methods and compare their performance. This work will focus on investigating various matrix decomposition algorithms— such as NMF, SVD, and SGD — to predict missing movie ratings.

The dataset contains approximately 100,000 ratings – about 600 users have rated around 10,000 movies. `userId` is a unique user identifier, `movieId` is a unique movie identifier, and rating is a rating from 0 to 5 (ratings may also include half points, e.g., 3.5, 4.5). The `timestamp` field will not be used.

The data will be transformed into matrix form. Let Z be a matrix containing the training ratings – a matrix of size n × d, where n is the number of users and d is the number of movies. The implementation of NMF requires a fully populated matrix Z. Therefore, missing values must first be imputed. Missing values will be filled with the weighted mean of average rating of the corresponding movie and average rating given by the user in 1:1 ratio.

The predictions will be compared with ground truth, and a root-mean square error will be computed:

$$R = \sqrt{\frac{1}{|T|} \sum_{(u,m) \in T} (Z'[u,m] - T[u,m])^2}$$

where T is a given set,$Z'[u,m]$ is a rate prediction for user u and movie m, and $T[u,m]$ is the "true" rating.

2 Algorithms

2.1 Non-Negativ Matrix Factorization

Non-negative matrix factorization (NMF) is an algorithm that decomposes matrix Z of size n × d into matrices W of size n × r and H of size r × d:

$$Z \approx WH$$

where r will be approximated to choose the best parameter to get the smallest RMSE. If r is too small, WH might not have enough "flexibility" to approximate Z well, resulting in a poor approximation. On the other hand if r is too large, while WH could approximate Z very well, there model may be overfitted.

2.2 Singular Value Decomposition 1

Singular value decomposition (SVD) is a method of decomposing a matrix into three other matrices. If we have a matrix Z of size n × d, SVD decomposes it into:

$$Z \approx U \Sigma V^T$$

Where:

- U is a n × n matrix, and its columns are the left singular vectors of Z,
- Σ is a diagonal n × d matrix. The diagonal elements are the square roots of the eigenvalues of $Z^T Z$,
- V is a d × d matrix, and its columns are the right singular vectors of Z.

In this work a slightly modified version of this method - truncated SVD - will be used:

$$Z \approx U_r \Sigma_r V_r^T$$

We choose to keep only the top r singular values and corresponding singular vectors in Σ, U, and V. This gives us a lower-rank approximation of Z.

2.3 Singular Value Decomposition 2

While the initial SVD1 method relies on applying truncated Singular Value Decomposition to a matrix where missing values have been imputed beforehand, SVD2 follows a different approach for computing the SVD approximation. Instead of statically filling the empty spaces before factorization, this alternative procedure handles the sparse matrix structure differently. The exact formulation of this method adapts to the specific characteristics of the recommendation dataset, aiming to provide an alternative lower-rank approximation of Z that potentially mitigates the bias introduced by global or user/movie-specific mean imputations.

2.4 Stochastic Gradient Descent

In the previous methods, missing values in the matrix Z had to be imputed before applying factorization. The Stochastic Gradient Descent (SGD) approach reformulates the problem so that missing values are handled directly within the optimization process. For a given matrix Z of size $n \times d$, we seek a user feature matrix W of size $n \times r$ and a movie feature matrix H of size $r \times d$. The goal is to minimize the error exclusively on the known ratings. The objective function is defined as:

$$\arg\min_{W,H} \sum_{(i,j):z_{ij}\neq'?'} (z_{ij} - w_i^T h_j)^2 \tag{1}$$

To prevent the model from overfitting, a regularization term with parameter $\lambda > 0$ is introduced:

$$\arg\min_{W,H} \sum_{(i,j):z_{ij}\neq'?'} (z_{ij} - w_i^T h_j)^2 + \lambda(||w_i||^2 + ||h_j||^2) \tag{2}$$

where h_j is the j-th column of matrix H, and w_i^T is the i-th row of matrix W. Various optimization techniques can be used to minimize this objective function. In this context, gradient-based optimization algorithms, such as SGD or the Adam optimizer, iteratively update the matrices W and H to find the best possible approximation.

2.5 Hybrid Model

To leverage the strengths of both matrix decomposition and gradient-based optimization, a Hybrid Model was developed. This approach combines the global pattern recognition of Singular Value Decomposition (SVD) with the personalized, nuanced adjustments of Stochastic Gradient Descent (SGD).

The model generates a final prediction $\hat{z}_{ij}$ for user i and movie j by blending the baseline prediction from the truncated SVD with the prediction from the SGD model using a weighting hyperparameter $\alpha \in [0, 1]$:

$$\hat{z}_{ij} = \alpha Z_{SVD}[i, j] + (1 - \alpha)(\mu + b_i + b_j + w_i^T h_j) \tag{3}$$

where:

- $Z_{SVD}[i, j]$ is the global baseline prediction obtained from the pre-computed truncated SVD matrix.
- μ is the global average of all known ratings.
- b_i and b_j represent the user and movie biases, capturing their systematic deviations from the global mean.
- $w_i^T h_j$ is the dot product of the user and movie feature vectors, extracting specific interactions.

During the training process, the SVD component remains static, while the biases (b_i, b_j) and feature matrices (W, H) are iteratively updated using stochastic gradient descent. The optimization minimizes the error between the known ratings z_{ij} and the hybrid predictions $\hat{z}_{ij}$, while applying regularization to prevent overfitting. A grid search is utilized to determine the optimal combination of the weight α, the number of latent factors r, the learning rate, and the regularization parameters.

3 Data Analysis

Let's start with analysing the given data. At the beginning the distribution of all the ratings will be drawn, so we will be able to better determine how to fill empty values in matrix Z.

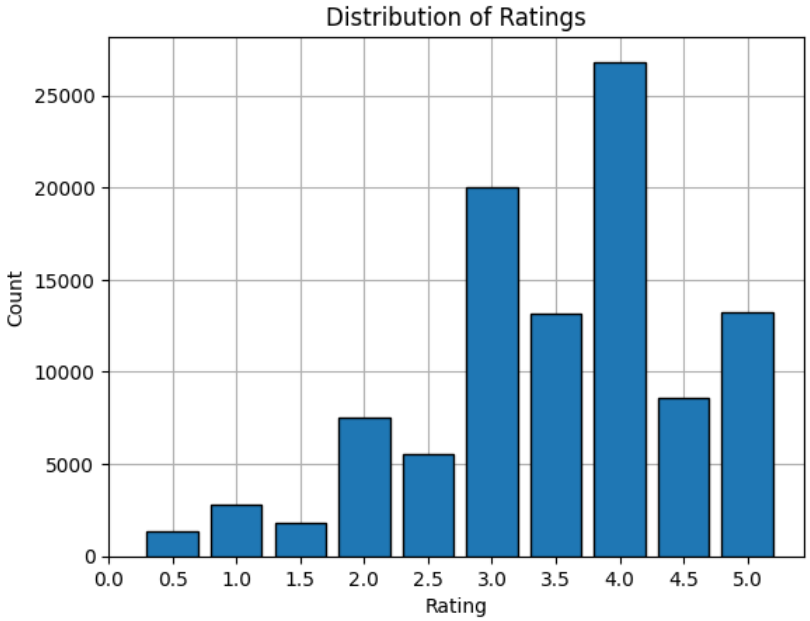

Figure 1: Distribution of all the ratings

As we can see, the distribution is inclined to the right, so the mean will be around 3.5. This means that most of the users gave positive ratings to the movies.

Now let's check the average movie rating for each user.

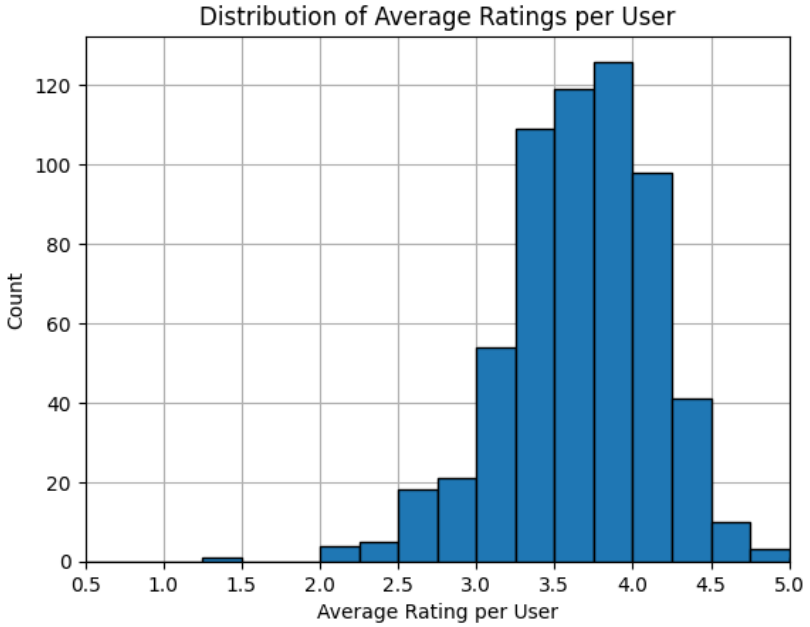

Figure 2: Distribution of average rating per user

We can see that this distribution is centered between $3.5 - 4.0$ meaning that the average rating given by the user is around this value.

At last, let's take a look at a distribution of average user rating per movie.

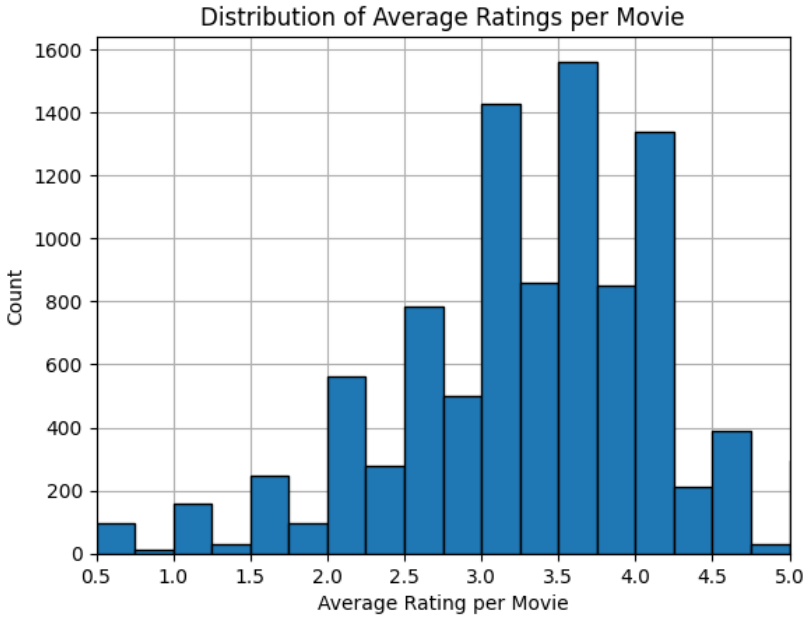

Figure 3: Distribution of average rating per movie

The distribution of average user rating per movie differ from the distribution of average movie rating per user. The mean is probably a bit smaller than in previous distribution. This leads to the conclusion that using it in matrix Z to fill the blanks will give different results.

The last thing we will look at will be the sparsity of the matrix Z. This will inform us on how many values we will have to fill in matrix Z.

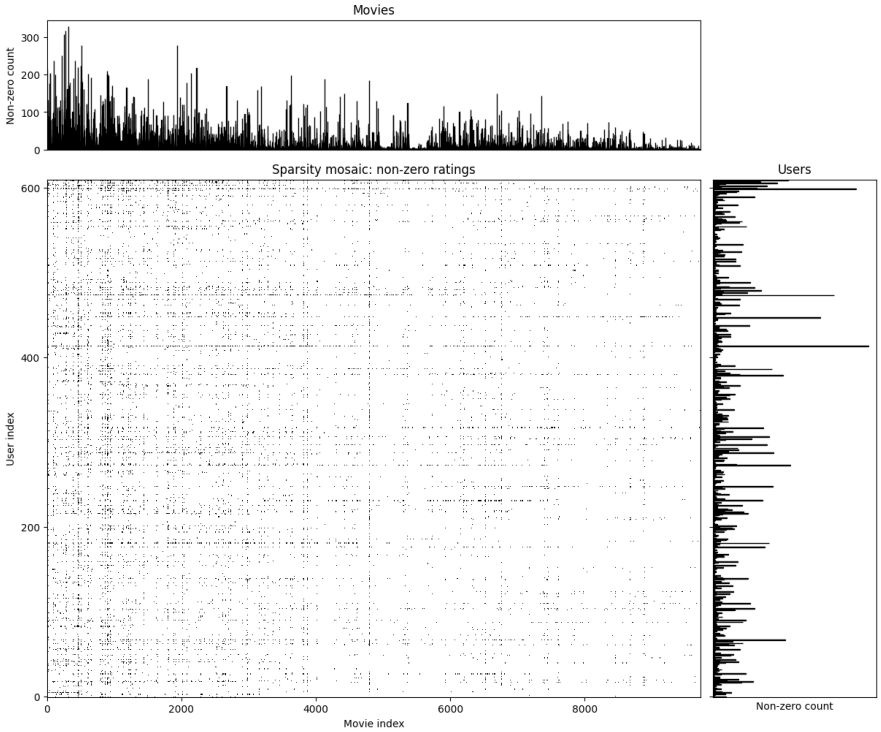

Figure 4: The sparsity of matrix Z

The white cells indicate the empty spaces and the black cells non-empty values. The sparsity of the matrix is equal to 98.3% meaning we have to fill 98.3% of empty values in this matrix.

4 The choice of the method of filling

To determine the best method of filling the empty values in Z matrix, four different approaches were compared:

- filling missing entries with 0,
- replacing missing values with average rating of the corresponding movie,
- replacing missing values with average rating given by the user,
- replacing missing values with the weighted mean of point 2 and 3 (in ratio 1:1).

To choose the best approach those four methods were compared by NMF algorythm with $r \in \{10, 25, 50\}$. The results are in the table below.

Table 1: Model Performance: RMSE by Strategy and Rank (r)

Rank (r)	**Zero**	**Movie Mean**	**User Mean**	**Weighted**
10	2.4968	0.8482	0.9020	0.8275
25	2.5286	0.8482	0.9053	**0.8272**
50	2.5631	0.8510	0.9115	0.8298

Overall the best results are for the weighted strategy. The RMSE on training set is around 8.27 which is the lowest of all the methods. Similar but a bit worse results appear for the movie mean. The worst numbers are when we fill all the empty spaces with zeros, which intuitevely makes sense. RMSE here is around 2.5, that is 3 times higher than while using the weighted strategy.

According to these results we will be using the weighted strategy to determine the best RMSE in the algorithms.

5 Models performance and comparison of the results

5.1 NMF

To get the smallest RMSE possible, the performance of the models was checked for different values of r. The best results were achieved for $r \in [10, 30]$. The RMSE on training set was the lowest and the results on a test set were the best. The graph below illustrates how model perform on a training data set in terms of RMSE.

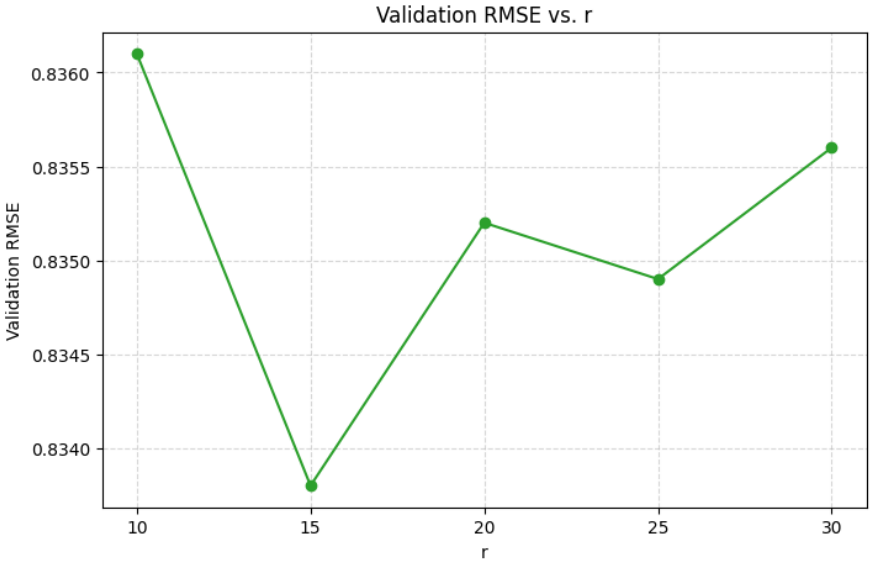

Figure 5: RMSE for NMF for different r

The best results were achieved for $r = 15$ as the RMSE was the lowest in this particular choice. On the test set the RMSE was a bit higher (around 0.891) than on training set which can point to the possibility of overfitting the model. For other values of r the results on the test set were pretty similar, so we will assume that's the acceptable difference. The smallest difference between datasets was for approximately $r = 25$ where RMSE on training set was around 0.835 and on test set it was around 0.889.

5.2 SVD1

Similarly like for NMF, to obtain the smallest RMSE possible, different values of r were tested. The best results were achieved for the similar range of r as before (around $r \in [10, 50]$), which is visible on the graph below.

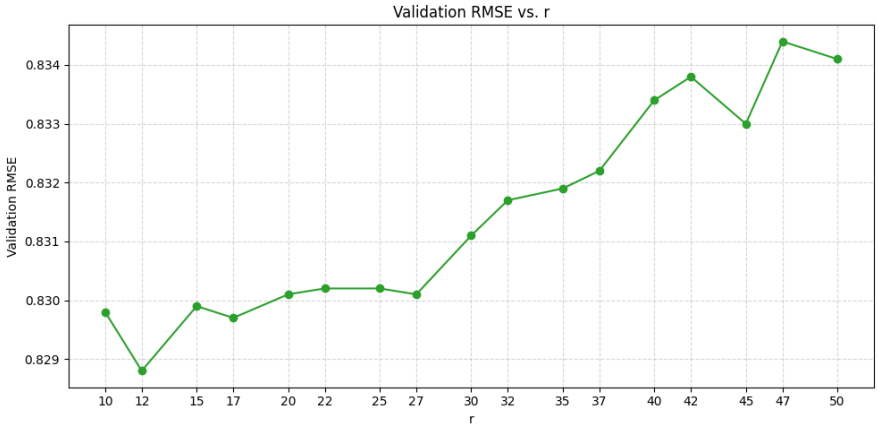

Figure 6: RMSE for SVD1 for different r

On the the train set the lowest RMSE was achieved for $r = 12$ and it is equal to around 0.829, but for the test set the RMSE is much higher and equal to around 0.894. This can point to a model being overfitted. The sweet spot seems to be around $r = 50$, where on a training set RMSE is equal to 0.834, but on test set it is around 0.894, which reduces the difference between train and test sets.

5.3 SVD2

The SVD2 method was tested for various values of the rank parameter $r \in [10, 60]$. As shown in Figure 7, the validation RMSE remains relatively stable, oscillating around 0.83.

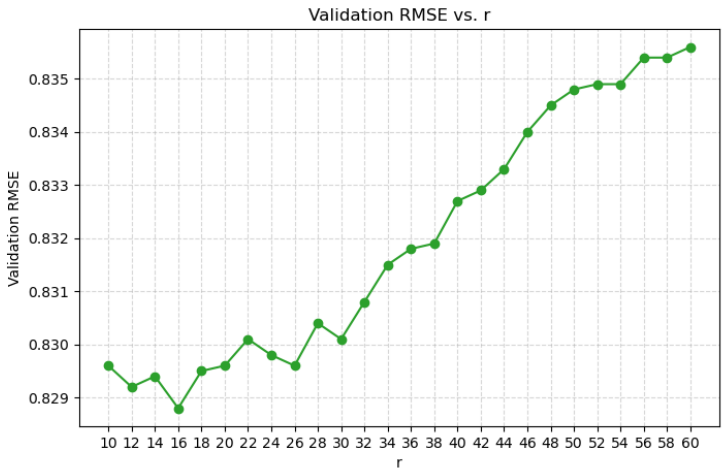

Figure 7: RMSE for SVD2 for different r

The hyperparameter tuning process identified $r = 16$ as the optimal rank, achieving a validation RMSE of 0.8288. When evaluated on the final test set, the model reached an RMSE of 0.8909 and a Mean Absolute Error (MAE) of 0.6533. The R^2 coefficient was 0.2870. These results suggest that while SVD2 is effective, there is a noticeable gap between validation and test performance, which may indicate a slight sensitivity to the data distribution.

5.4 SGD

For the Stochastic Gradient Descent approach, the influence of the latent factor dimension r was similarly evaluated. The results for different r values are illustrated in Figure 8.

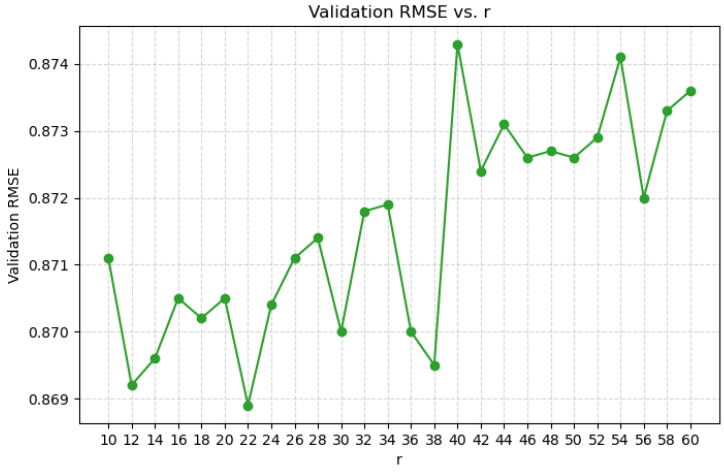

Figure 8: RMSE for SGD for different r

The best validation performance was achieved at $r = 22$ with an RMSE of 0.8689. During the training process for the best model (30 epochs), the training RMSE decreased significantly from 0.9598 to 0.6820, showing that the model effectively learns the underlying patterns in the rating matrix. On

the test set, the SGD model achieved an RMSE of 0.8554 and an MAE of 0.6300. With an R^2 of 0.3427, the SGD approach provided the most accurate predictions among the discussed methods, demonstrating the advantage of optimizing the loss function directly on observed ratings.

5.5 BEST (Hybrid)

The performance of the BEST hybrid model was evaluated across a grid of hyperparameters, specifically focusing on the number of latent factors ($n\_factors$) and the weight assigned to the SVD baseline (svd_weight).

Figure 9 presents a heatmap of the validation RMSE for various parameter combinations. It clearly shows the regions where the model achieves its optimal performance, emphasizing the importance of finding the right balance between the matrix factorization and gradient descent components.

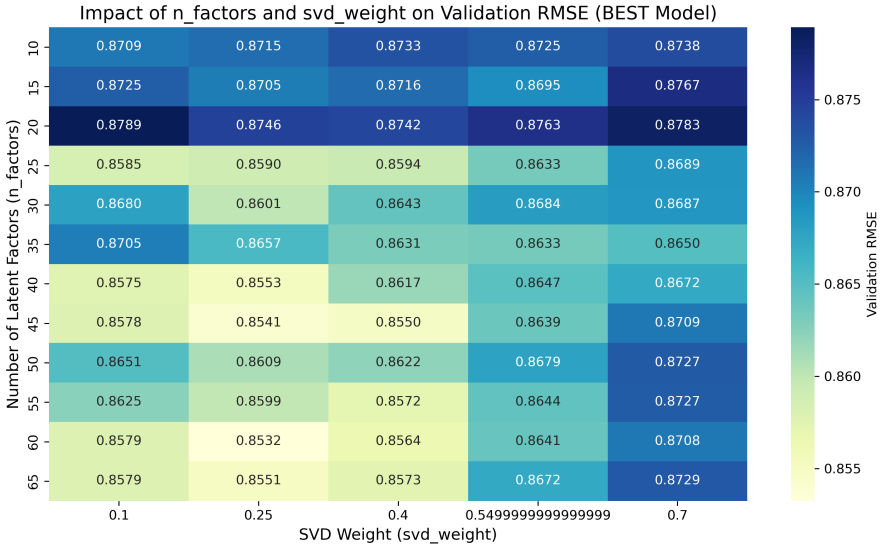

Figure 9: Impact of $n\_factors$ and svd_weight on Validation RMSE.

To further illustrate this relationship, Figure 10 demonstrates how the validation RMSE changes as the SVD weight increases, plotted for different numbers of latent factors. The "U-shape" of these curves confirms the underlying hypothesis of the hybrid approach: a blend of both global trends (SVD) and localized user-item interactions (SGD) yields a lower error rate than relying on either method independently. The model reaches its peak performance when the SVD weight is carefully balanced, avoiding the extremes of 0.0 and 1.0.

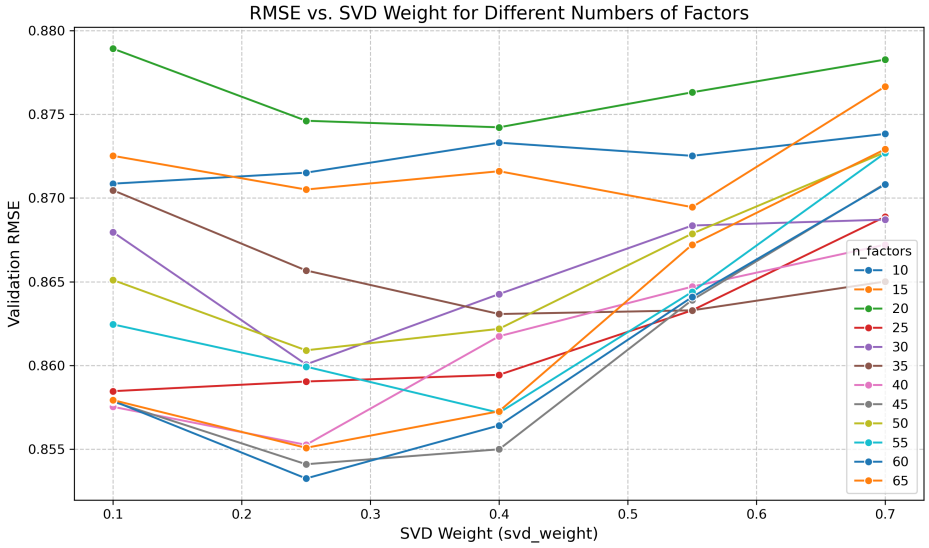

Figure 10: RMSE vs. SVD Weight for different numbers of latent factors.

6 Validation

To reduce data leakage the validation set was separated from the set. Then the models performance was computed on this dataset and compared using three different metrics: RMSE, R^2 and MAE (mean absolute error).

Table 2: Models Performance Comparison

Model	**RMSE**	R^2	**MAE**
NME	0.8902	0.2622	0.6681
SVD1	0.8891	0.264	0.6672
SVD2	0.8899	0.2627	0.6683
SGD	0.8830	0.2741	0.6576

As we can observe, the best results for every researched metrics were achieved using Stochastic Gradient Decent method. We were able to get the smallest RMSE and MAE so that means the prediction were the closest to the real data out of all applied methods. The R^2 result was the highest which shows that out of every attempt, in this one the model was the best fitted to the data.

The results for the rest of the methods were pretty similar. The worst numbers were achieved for NME, which apart from that was also the slowest performing model. In conclusion using it gives the worst result under every condition. SVD1 and SVD2 were pretty similar and gave satisfactory results.

Overall Top Performing Models

To conclude the evaluation, Figure 11 visualizes the top 10 best-performing model configurations across all experiments based on their test RMSE. As illustrated, the BEST (Hybrid) model variants consistently dominate the top rankings, proving that the integration of SVD and SGD significantly outperforms the standalone implementations like NMF or purely SVD-based approaches.

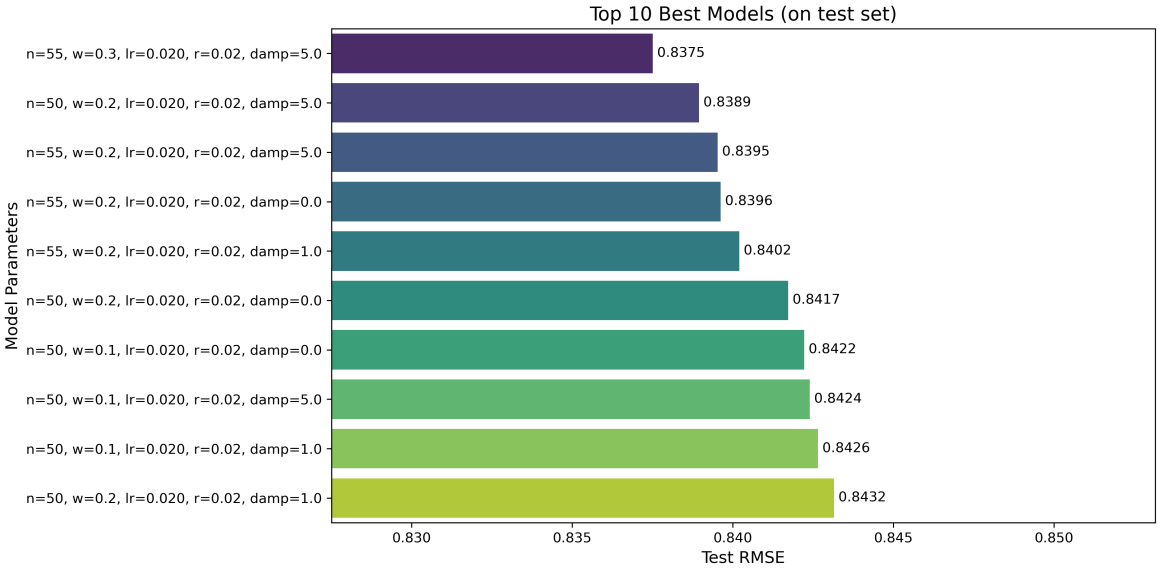

Figure 11: Top 10 best-performing models evaluated on the test set.

7 Summary

This project successfully implemented and evaluated five distinct recommendation strategies to predict movie ratings based on a dataset of approximately 100,000 entries. A critical finding of the initial data analysis was the high sparsity of the user-movie matrix (98.3%). Some tests revealed that a **weighted mean imputation**, combining movie and user averages in a 1:1 ratio, provided the most reliable approach for matrix decomposition compared to zero-filling or mean imputation.

Across most models, the rank r achieved the best performance within the range of 10 to 30. Higher values tended to cause overfitting. The **Stochastic Gradient Descent (SGD)** turned out to be the best method. By optimizing the loss function exclusively on observed ratings and incorporating regularization, it achieved the lowest **RMSE (0.8830)** and the highest R^2 **(0.2741)** on the test set.

While **NMF** and **SVD** variants provided satisfactory results, they were more sensitive to the imputation bias and generally performed worse than SGD. Additionally, NMF was noted as the slowest performing model in this study.

In conclusion, for sparse datasets like the one analyzed, **direct optimization via SGD** is more effective than traditional decomposition methods that require global imputation prior to factorization.